

PolyFlop8

Block Specification Manual

Component Interfaces & Internal Logic

Revision: 1.2
Date: January 6, 2026

1 REVISION HISTORY

Revision	Change	Date
1.0	Initial document creation	04.01.2026
1.1	Change general architecture. Add ALU in to DRAM addr mux.	05.01.2026
1.2	Change RegFile addr_b to accept 8-bit and slice bus internally due to architectural decision	06.01.2026

2 POLYFLOP8 GENERAL ARCHITECTURE

The PolyFlop8 is a simple 8-bit microcontroller implemented in synthesizable VHDL for FPGA targets. Maybe it has a 1970s vibe, but at least it's also unoptimized and sluggish.

The processor follows a strict **Harvard Architecture** with separate Program ROM and Data RAM. The simple 2-stage pipeline allows simultaneous instruction fetch and execution.

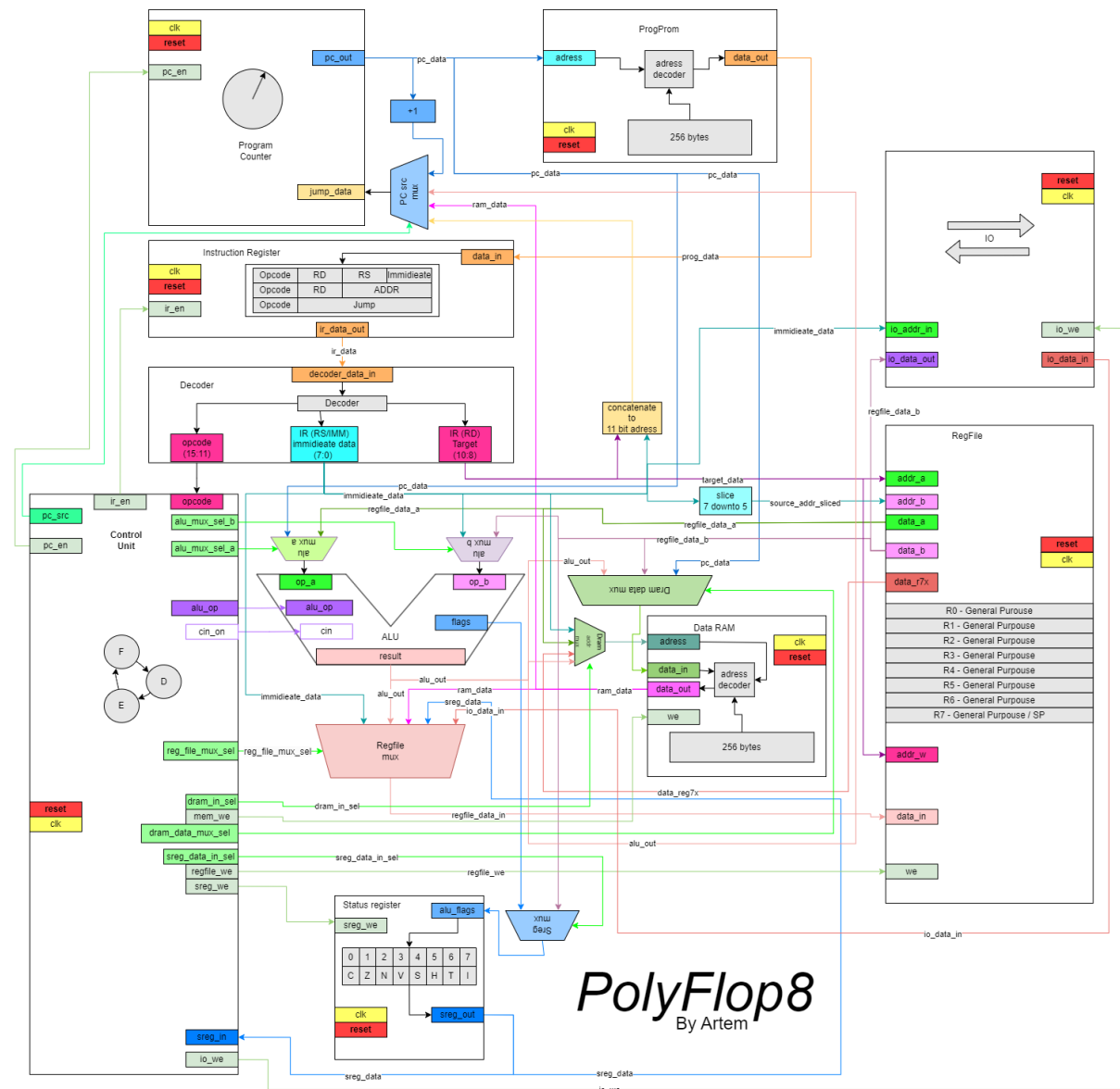


Figure 1: PolyFlop8 processor

3 ALU (ARITHMETIC LOGIC UNIT)

The ALU is a combinational block responsible for arithmetic and logical operations. It features internal multiplexers for operand selection and a flag generation unit.

3.1 Implementation Schematic

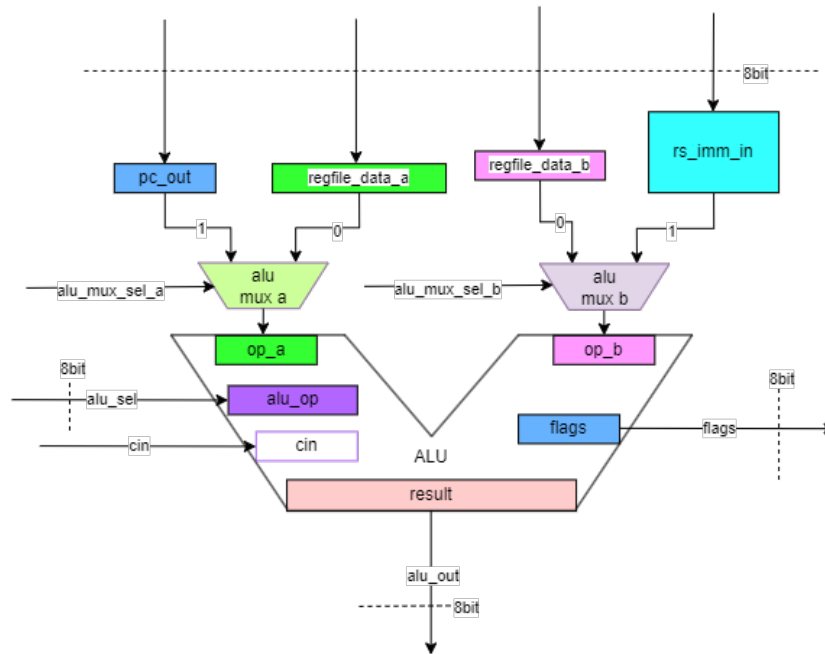


Figure 2: ALU

3.2 Interface Signals

Signal Name	Dir	Width	Description
pc_data	IN	8-bit	Data from Program Counter (Channel A alt input)
regfile_data_a	IN	8-bit	Data from Register File A (Channel A default)
rs_imm_in	IN	8-bit	Immediate value or Source Address (Channel B alt)
regfile_data_b	IN	8-bit	Data from Register File B (Channel B default)
alu_mux_sel_a	IN	1-bit	Mux A Select: '0'=RegFile A, '1'=PC
alu_mux_sel_b	IN	1-bit	Mux B Select: '0'=RegFile B, '1'=Imm/RS
alu_sel	IN	4-bit	Operation Opcode (See Table 3.3)
cin	IN	1-bit	Carry Input (for ADC/SBC)
result	OUT	8-bit	Result of the operation
flags	OUT	5-bit	Status Flags: H(4), V(3), N(2), C(1), Z(0)

Table 1: ALU Entity Port Definition

3.3 Operations Table (`alu_sel`)

Code	Name	Operation	Flags Affected
0000	ADD	$A + B$	H, V, C, N, Z
0001	ADC	$A + B + C_{in}$	H, V, C, N, Z
0010	SUB	$A - B$	V, C, N, Z
0011	SBC	$A - B - C_{in}$	V, C, N, Z
0100	AND	$A \wedge B$	Z, N
0101	OR	$A \vee B$	Z, N
0110	XOR	$A \oplus B$	Z, N
0111	MOV	Pass B	None
1000	LDI	Pass A	None
1001	COM	$\neg A$ (1's complement)	C=1, Z, N
1010	NEG	$0 - A$ (2's complement)	C, V, Z, N

4 INSTRUCTION DECODER

The Decoder is a purely combinational block that slices the 16-bit instruction word into usable control signals and immediate values.

4.1 Implementation Schematic

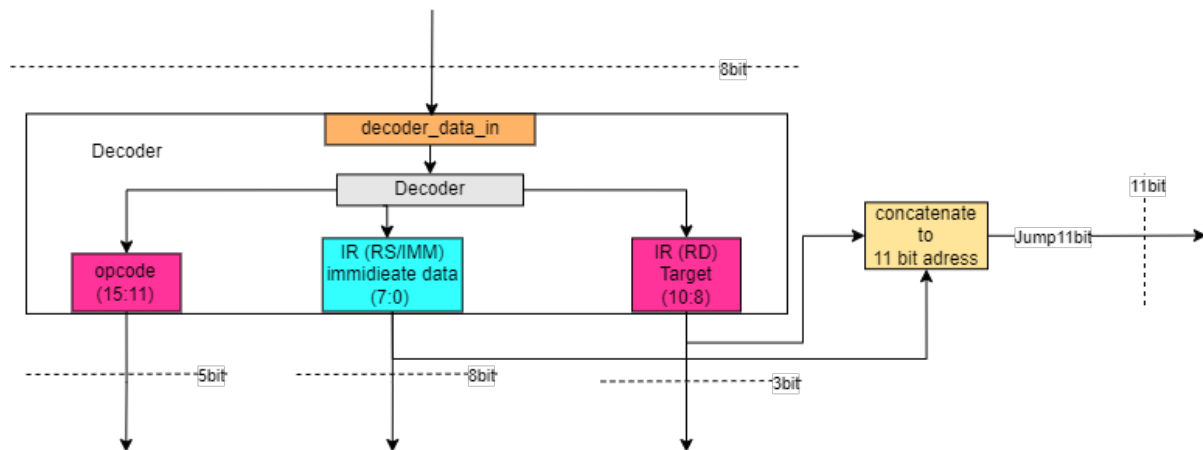


Figure 3: Decoder

4.2 Interface Signals

Signal Name	Dir	Width	Description
decoder_data_in	IN	16-bit	Full instruction word fetched from ROM
opcode	OUT	5-bit	Instruction Opcode (Bits 15 down to 11)
RD	OUT	3-bit	Destination Register Address (Bits 10 down to 8)
RS_IMM	OUT	8-bit	Combined field for Source Reg or Immediate (Bits 7 down to 0)
Jump11bit	OUT	11-bit	Absolute address for JMP/CALL (Bits 10 down to 0)

Table 2: Decoder Entity Port Definition

4.3 Functional Logic

- **Slice 1 [15:11]:** Opcode. Used by Control Unit to determine state.
- **Slice 2 [10:8]:** Typically Rd (Destination Register).
- **Slice 3 [7:0]:** RS_IMM. This field is polymorphic; the Control Unit determines if bits [7:5] are treated as Rs (Source Register) or if the whole byte is an Immediate value.
- **Slice 4 [10:0]:** Jump11bit. Overlaps with RD and RS_IMM. Used only during Jump/Call operations.

5 PROGRAM COUNTER (PC)

The PC Unit manages the address of the next instruction. It contains an 11-bit register and a 4-way multiplexer to handle sequential execution, jumps, branches, and returns.

5.1 Implementation Schematic

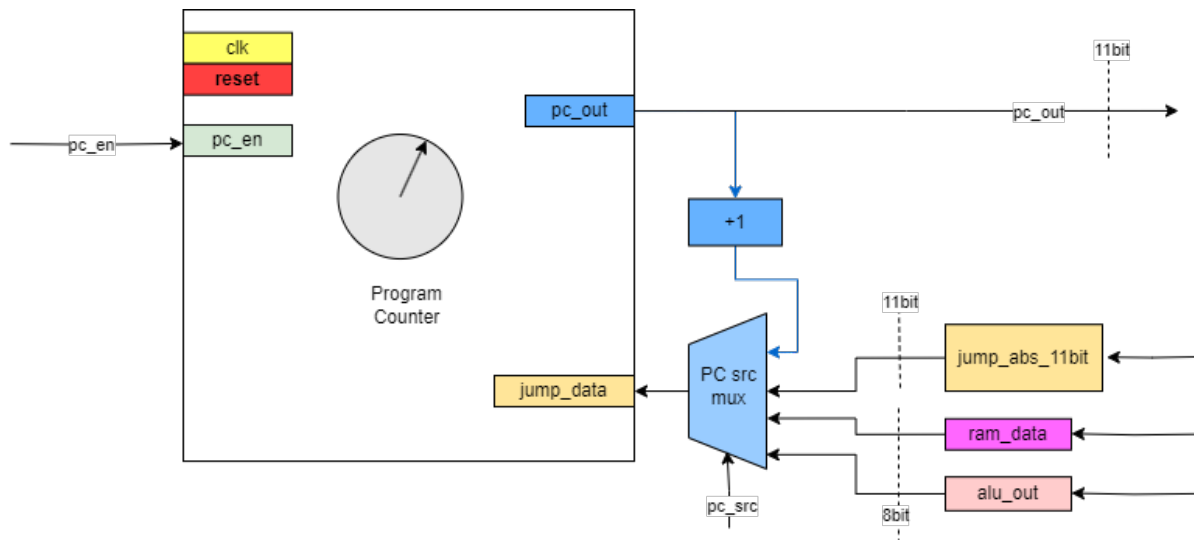


Figure 4: Program Counter

5.2 Interface Signals

Signal Name	Dir	Width	Description
clk, rst	IN	1-bit	System Clock and Active High Reset
pc_en	IN	1-bit	PC Write Enable (Stall control)
pc_src	IN	2-bit	Source Selector (See Table 5.3)
alu_out	IN	8-bit	Signed offset for Relative Branching
ram_data	IN	8-bit	Return address lower byte (Indirect/RET)
jump_abs_11bit	IN	11-bit	Target address for Absolute Jumps
pc_out	OUT	11-bit	Current Program Counter Address

Table 3: ProgramCounter Entity Port Definition

5.3 Next Address Logic (`pc_src`)

<code>pc_src</code>	Mode	Logic Description
00	Fetch	$PC \leftarrow PC + 1$ (Sequential execution)
01	Branch	$PC \leftarrow (PC + 1) + \text{sign_extend}(\text{alu_out})$ Performs relative jump using signed 8-bit offset.
10	Indirect/Ret	$PC \leftarrow \text{resize}(\text{ram_data})$ Loads address from Memory/Stack. Note: Currently supports page 0 (8-bit) only.
11	Jump	$PC \leftarrow \text{jump_abs_11bit}$ Loads absolute 11-bit address from instruction.

6 REGISTER FILE

The Register File contains 8 general-purpose 8-bit registers (R0-R7). It supports two asynchronous read ports and one synchronous write port with multiple input sources.

6.1 Implementation Schematic

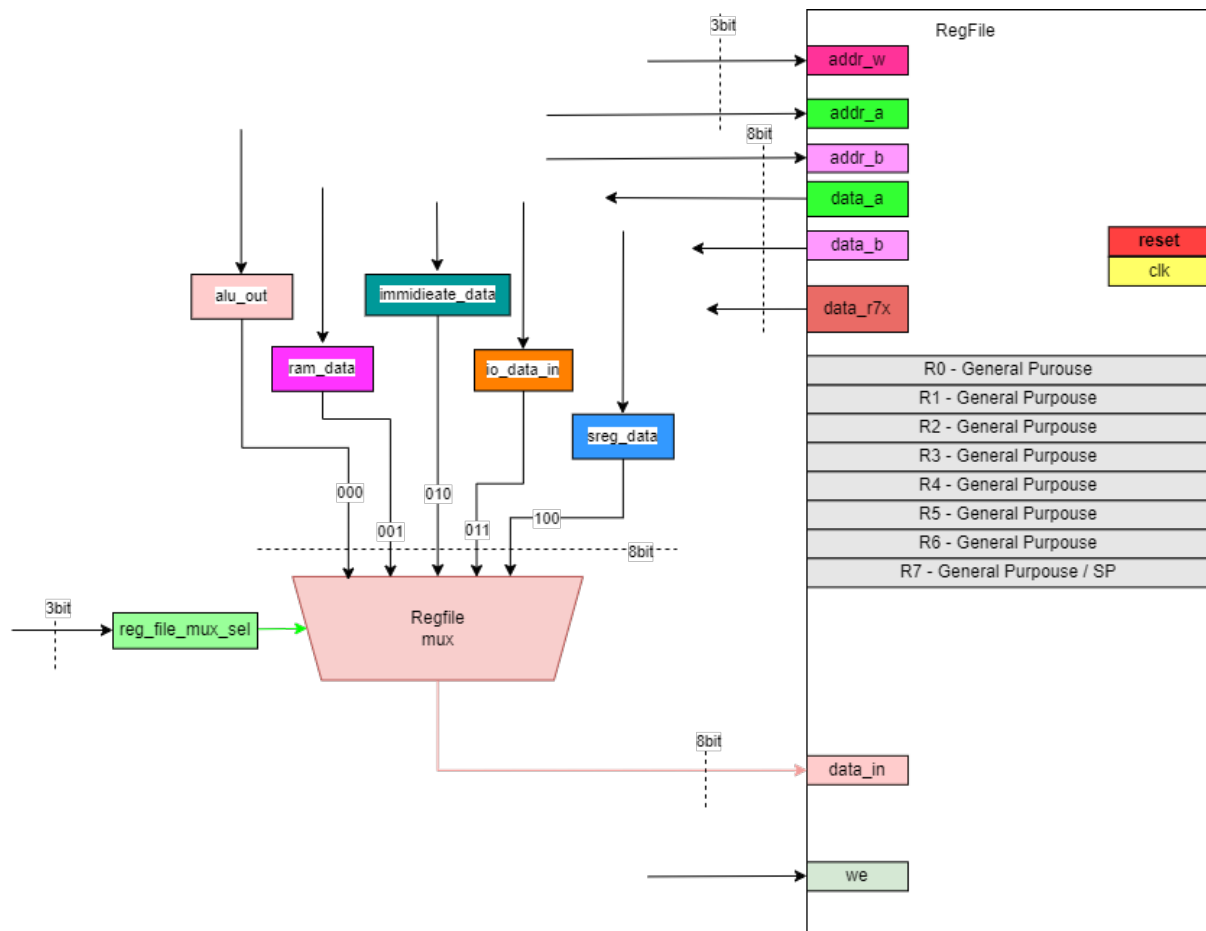


Figure 5: Register File

6.2 Special Functions

- **R7 (X-Pointer):** Register 7 has a dedicated output port `data_r7x`. This allows the Memory Unit to use R7 directly as an address pointer for Indirect Addressing modes without using the ALU.
- **Write Mux:** The `rf_src_sel` signal determines which input bus is written to the destination register when `we` is high.

6.3 Interface Signals

Signal Name	Dir	Width	Description
clk, rst	IN	1-bit	System Clock and Active High Reset
we	IN	1-bit	Write Enable
addr_a	IN	3-bit	Read Address Port A
addr_b	IN	8-bit	Read Address Port B (internal slicing)
addr_w	IN	3-bit	Write Address Port
rf_src_sel	IN	3-bit	Write-Back Source Selector
Write-Back Data Inputs			
alu_out	IN	8-bit	ALU Result
ram_data	IN	8-bit	Data from RAM
rs_imm_in	IN	8-bit	Immediate Value
io_data_in	IN	8-bit	I/O Port Data
sreg_data	IN	8-bit	Status Register Data
Outputs			
data_a	OUT	8-bit	Output Data Port A
data_b	OUT	8-bit	Output Data Port B
data_r7x	OUT	8-bit	Hardwired Output of Register R7 (Index Pointer)

Table 4: RegFile Entity Port Definition

7 INSTRUCTION REGISTER (IR)

The Instruction Register acts as a stable buffer between the Program Memory (PROM) and the Decoder. It latches the 16-bit instruction word to ensure signal stability during the decoding phase.

7.1 Implementation Schematic

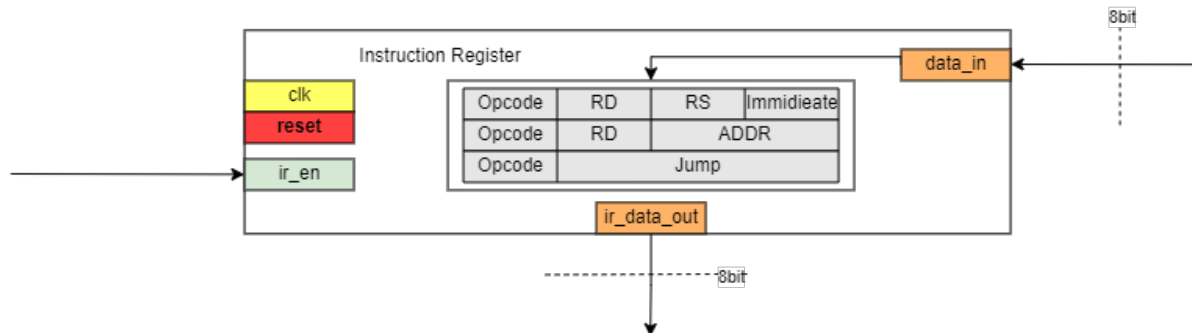


Figure 6: Instruction Register

7.2 Interface Signals

Signal Name	Dir	Width	Description
clk, rst	IN	1-bit	System Clock and Active High Reset
ir_en	IN	1-bit	Write Enable (Active during Fetch state)
prom_data	IN	16-bit	Raw instruction from Program Memory
ir_out	OUT	16-bit	Buffered instruction to Decoder

Table 5: Instruction Register Port Definition

8 CONTROL UNIT (CU)

The Control Unit is the central Finite State Machine (FSM) driving the processor's datapath. It generates specific control signals for the ALU, Register File, and RAM multiplexers.

8.1 Implementation Schematic

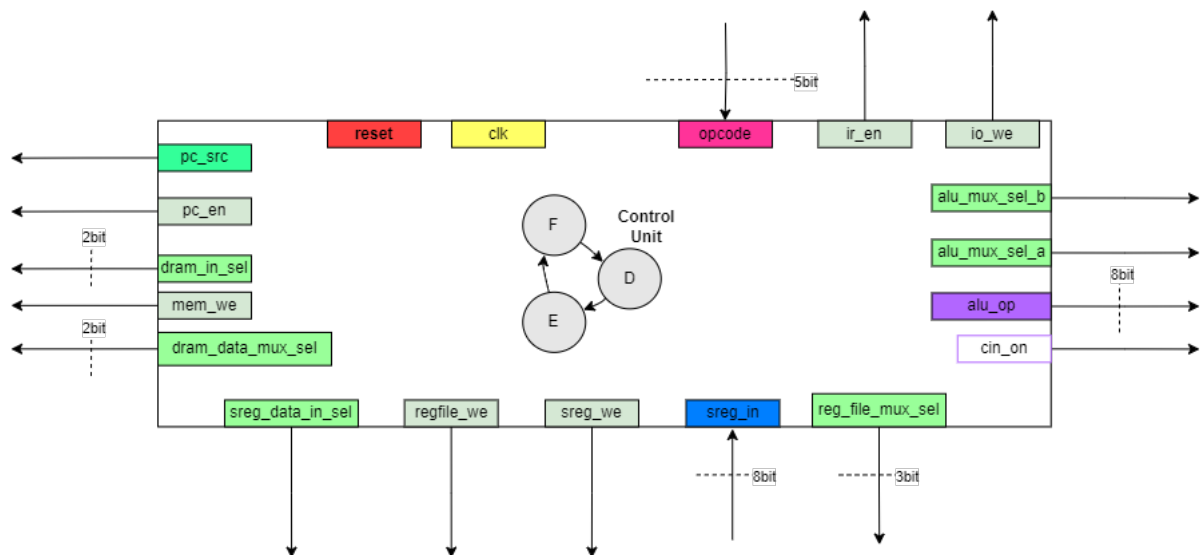


Figure 7: Control Unit

8.2 Interface Signals (Derived from Datapath Components)

Signal Name	Dir	Width	Description
opcode	IN	5-bit	Opcode from Decoder (Bits 15:11)
sreg_data	IN	8-bit	Status flags for conditional branching
alu_sel	OUT	4-bit	ALU Operation Opcode
rf_src_sel	OUT	3-bit	RegFile Write-Back Source (0-4)
mem_addr_sel	OUT	2-bit	RAM Address Source Selector
mem_data_sel	OUT	2-bit	RAM Data Source Selector
pc_src	OUT	2-bit	PC Next Address Mode
mem_we	OUT	1-bit	Data RAM Write Enable
regfile_we	OUT	1-bit	Register File Write Enable
sreg_we	OUT	1-bit	Status Register Write Enable
io_we	OUT	1-bit	I/O Write Enable

Table 6: Control Unit Control Signals

9 STATUS REGISTER (STATUSREG)

The Status Register stores arithmetic flags updated by the ALU or restored from the Register File. The logic includes dynamic calculation for the Sign flag ($S = N \oplus V$).

9.1 Implementation Schematic

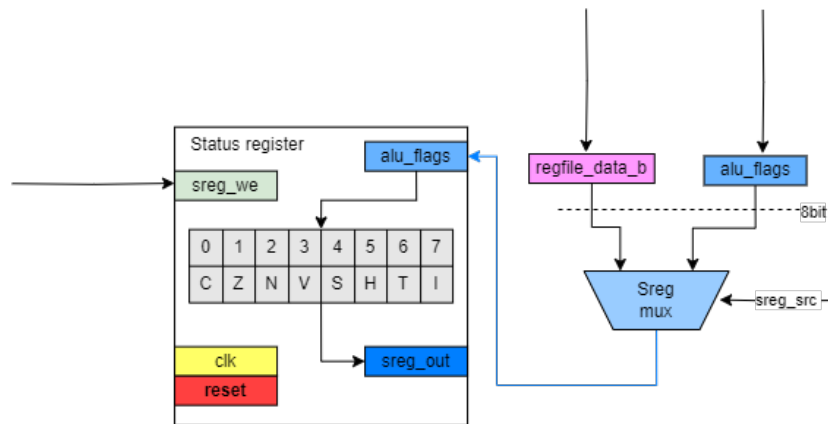


Figure 8: Status Register Logic

9.2 Internal Flag Mapping

Based on the VHDL architecture Behavioral, the flags are mapped as follows:

Bit	Flag	Source (ALU Mode)	Description
0	C	alu_flags(1)	Carry Flag
1	Z	alu_flags(0)	Zero Flag
2	N	alu_flags(2)	Negative Flag
3	V	alu_flags(3)	Overflow Flag (2's Comp)
4	S	$N \oplus V$	Sign Flag (Calculated)
5	H	alu_flags(4)	Half-Carry (BCD)
6	T	Latch	Bit Copy / Transfer (Preserved)
7	I	Latch	Interrupt Enable (Preserved)

9.3 Multiplexer Logic (sreg_src)

- **'0' (Math Mode):** Updates bits 0-5 from ALU flags; preserves bits 6-7.
- **'1' (Restore Mode):** Loads full 8-bit byte from regfile_data_b (e.g., POP SREG).

10 DATA RAM

The Data RAM entity integrates a 256-byte storage array with input multiplexers for address and data buses.

10.1 Implementation Schematic

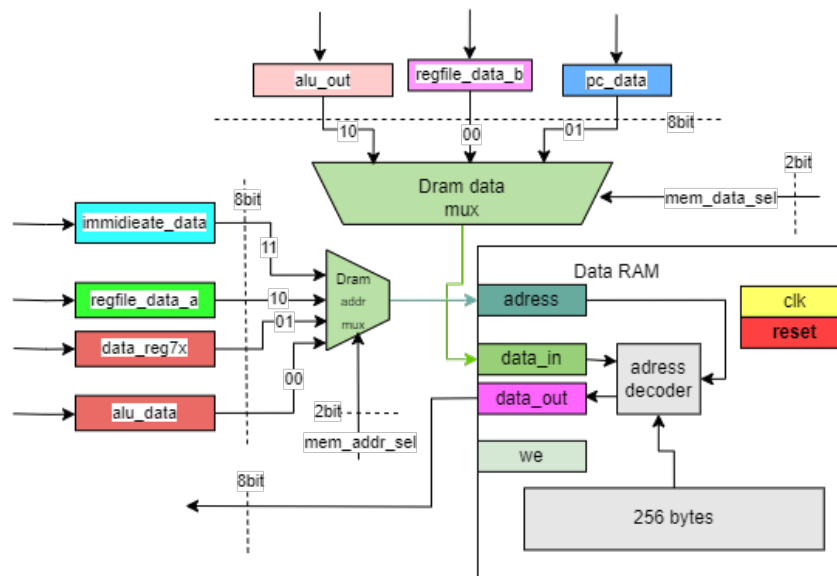


Figure 9: Data RAM

10.2 Address Multiplexer (`mem_addr_sel`)

Selects the pointer used to access memory.

Sel	Signal Source	Addressing Mode
00	alu_out	Base + Offset / Computed Address
01	data_reg7x	Indirect via R7 (X-Pointer)
10	regfile_data_a	Indirect via Register A
11	rs_imm_data	Direct / Immediate Address

10.3 Data Input Multiplexer (`mem_data_sel`)

Selects the data written to RAM when `mem_we = '1'`.

Sel	Signal Source	Operation
00	regfile_data_b	Store Register (ST)
01	pc_data	Store Return Address (CALL)
10	alu_out	Store Calculation (ALU Write)

11 I/O UNIT

The I/O Unit handles communication with external peripherals using **Port-Mapped (Isolated) I/O**. Unlike Memory Mapped I/O, this unit uses a dedicated address space separate from the Data RAM, allowing the use of the full 8-bit immediate field from the Instruction Register as a port address.

11.1 Implementation Schematic

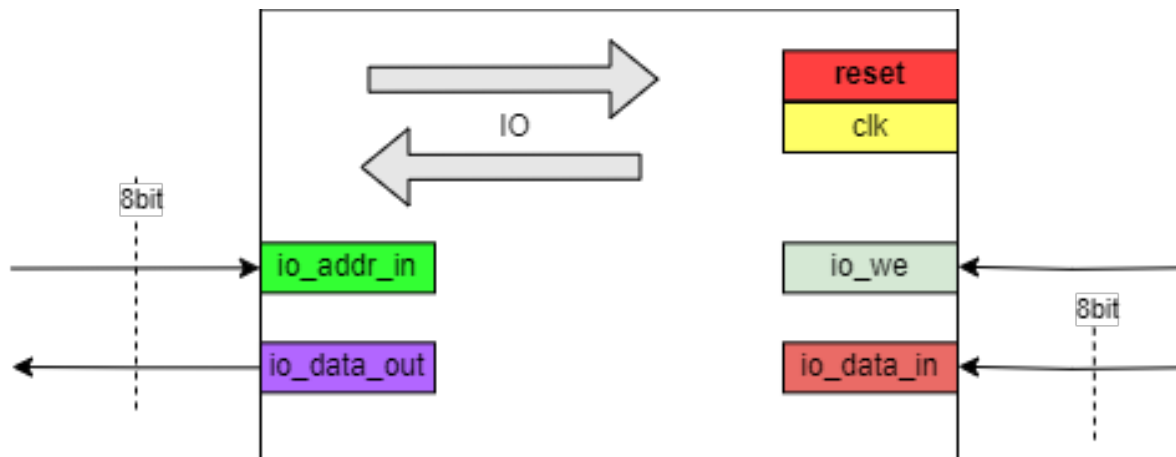


Figure 10: I/O Unit

11.2 Interface Signals

Signal Name	Dir	Width	Description
clk, rst	IN	1-bit	System Clock and Active High Reset
io_we	IN	1-bit	Write Enable (Strobe)
io_addr_in	IN	8-bit	Port Address (Derived from IR Immediate)
io_data_out	IN	8-bit	Data from CPU (to be written to Output)
io_data_in	OUT	8-bit	Data to CPU (read from Input)
port_out_a	OUT	8-bit	Physical Output Pins (e.g., LEDs)
pin_in_b	IN	8-bit	Physical Input Pins (e.g., Switches)

Table 7: I/O Unit Entity Port Definition

11.3 I/O Address Map

The I/O unit decodes the `io_addr_in` bus to select the active peripheral. The address space is 8-bit (0x00 to 0xFF).

Address (Hex)	Target Register	Function
0x01	reg_port_a	Write: Latch data to Port A Output
0x02	pin_in_b	Read: Sample data from Port B Input
Others	N/A	Reserved

11.4 Functional Logic

- **Output Operation (OUT):** When `io_we` is high and `io_addr_in` matches 0x01, the data from the internal `io_data_out` bus is latched into `reg_port_a`, driving `port_out_a`.
- **Input Operation (IN):** The unit acts as a combinational mux for reads. When `io_addr_in` matches 0x02, the state of physical pins `pin_in_b` is driven onto the `io_data_in` bus.
- **Integration Note:** The `io_data_in` bus connects to the Register File Input Mux (Select 011), while `io_addr_in` connects directly to the Instruction Register's immediate field (`IR[7:0]`).